Parameter-Efficient Fine-Tuning of SmolVLM-500M-Instruct for Visual Multiple-Choice Science QA

Chengkai Kang
ck4193@nyu.edu

Abstract

We present a parameter-efficient fine-tuning study of SmolVLM-500M-Instruct on a visual multiple-choice science question-answering task derived from ScienceQA. Working under a budget of at most 5 M trainable parameters and one Tesla T4 GPU, we adapt the model with Low-Rank Adaptation (LoRA) and conduct thirteen training runs that ablate capacity, training duration, LoRA target topology, prompt format, training objective, and inference-time interventions. Our strongest single-adapter submission reaches 0.831 on the public test leaderboard, while a deliberately "standard" inference recipe with exponential moving averaging and test-time augmentation scored only 0.805 on the same trained adapter, which is a 2.6-point regression we trace to under-warmed EMA shadow weights. We additionally show that two natural-looking interventions, generative chain-of-thought training and self-generated image captions, fail to improve over a clean letter-prediction baseline at this model scale. We release code, per-version configuration, and full ablation logs to support reproducibility.

1 Introduction

Vision-language models (VLMs) have rapidly improved on image-grounded reasoning benchmarks such as ScienceQA (Lu et al., 2022). State-of-the-art systems on these benchmarks typically combine large-capacity backbones (7 B+ parameters) with chain-of-thought (CoT) supervision (Wei et al., 2023). In contrast, deployment in resource-constrained settings often demands compact backbones, lightweight adaptation, and modest training compute.

This work investigates how far a 500 M-parameter VLM can be pushed on a multimodal science MCQ task using parameter-efficient fine-tuning. We work under three explicit constraints inherited from a course-organized competition: (1) the base model must be SmolVLM-500M-Instruct (Marafioti et al., 2025) from the official Hugging Face checkpoint; (2) at most 5 M trainable parameters; (3) all training and inference must run on a single Tesla T4 GPU available via Google Colab Free.

Within these constraints we performed thirteen training runs that systematically vary one or two factors at a time. Our contributions are:

- A best single-adapter score of **0.831** on the public test leaderboard, obtained with rank-24 LoRA on attention projections, five training epochs, and a deliberately minimal inference pipeline.
- Empirical evidence that two common "polish" interventions, exponential moving average (EMA) of LoRA weights and test-time augmentation by choice-order permutation (TTA), silently cost 2–3 percentage points on this task. We trace the EMA regression to insufficient warm-up of the shadow weights given the run length.
- A negative result for generative chain-of-thought fine-tuning at this model scale: both an answer-first variant and a reasoning-first variant regress relative to a single-token letter-prediction baseline, by 6.2 and 7.2 points respectively.
- Modest evidence that self-generated image captions combined with mixed attention-and-MLP LoRA targets recover competitive accuracy (0.823) but do not exceed the simpler attention-only baseline.
- Distributional analysis of the provided splits, identifying a structural validation-to-test gap driven by differences in question-length and number-of-choices distributions.

We additionally make the experimental record (per-version configuration, training metrics in a structured `metrics.json` format, and notebook-level code) available for reproduction.

2 Related Work

Multimodal MCQ benchmarks. ScienceQA (Lu et al., 2022) introduced grade-school science questions with image and text context, lecture material, and explanatory solutions. Subsequent multimodal MCQ benchmarks have substantially raised the difficulty bar; Yue et al. (2024) introduces 11,550 college-level questions across 30 subjects with mixed image, table, and diagram inputs, designed to require expert reasoning rather than retrieval.

Compact vision-language models. SmolVLM (Marafioti et al., 2025) is a compact VLM family with a SigLIP (Zhai et al., 2023) vision tower, a small language model decoder, and a learned modality-projection layer. Other open-weights compact VLM families include BLIP-2 (Li et al., 2023), which freezes the vision and language towers and trains only a Q-Former bridge, and the LLaVA-1.5 family (Liu et al., 2024a), which fine-tunes a 7 B+ language model on visual instruction-tuning data. SmolVLM is positioned as a much smaller alternative for resource-constrained deployment.

Parameter-efficient fine-tuning. LoRA (Hu et al., 2021) adapts a pretrained model by injecting low-rank trainable matrices into selected linear layers, leaving the original weights frozen. Several alternatives to LoRA exist within the parameter-efficient fine-tuning family. QLoRA (Dettmers et al., 2023) combines 4-bit base-model quantization with LoRA adapters; DoRA (Liu et al., 2024b) decomposes pretrained weights into magnitude and direction components and adapts only the latter; prefix-tuning (Li and Liang, 2021) prepends trainable continuous prompts in lieu of injecting low-rank matrices. We did not benchmark these against vanilla LoRA in this study, but flag QLoRA in particular as a likely candidate at the 2–7 B base-model scale that our limitations section identifies as the natural followup.

Chain-of-thought training. Wei et al. (2023) demonstrate that supervising models on full reasoning traces (rather than just final answers) improves multi-step reasoning at sufficient model scale. The literature suggests this benefit emerges around 60 B+ parameters and is unreliable at smaller scales (Wei et al., 2022); our negative result for generative CoT fine-tuning at 500 M is consistent with that account.

3 Task and Data

3.1 Task Formulation

The task is multiple-choice question answering with image grounding. Each example consists of (i) an image, (ii) a question text, (iii) optional supporting context (a `hint` passage and a `lecture` text), and (iv) a list of $k \in \{2, 3, 4, 5\}$ candidate answers represented as natural-language strings. The system must select the index of the correct answer. Training examples additionally contain a `solution` field which we use only as input context during training (Section 4); the test set does not contain solution text. We treat the prediction problem as conditional next-token classification over a fixed set of letter tokens `{A, B, C, D, E}`, masking out letters beyond the example's k.

3.2 Dataset

We use a course-provided ScienceQA-derived benchmark (Lu et al., 2022) with 5,165 examples total, partitioned into a training set of 3,109 examples, a validation set of 1,048 examples, and a test set of 1,008 examples (a 60/20/20 split). The splits were provided by the organizers and we used them as given without re-splitting. Test labels are withheld; predictions are scored on a public test-leaderboard accuracy metric.

Each row carries metadata fields (`subject` $\in$ {natural science, social science, language science}, `grade` from grade1 to grade12, `topic`, `category`, `skill`) which we use only for analysis, not for training. Images are PNG files with widely varying dimensions, ranging from 162×162 to 760×506 pixels in our sample inspection.

3.3 Validation-Split Role

The validation set serves three explicit roles in our experimental loop:

1. **Periodic evaluation during training.** Every 50 optimizer steps we compute validation accuracy, save the best-val LoRA adapter to disk, and record both the checkpoint step and the accuracy. This drives "best checkpoint" selection within a single run.

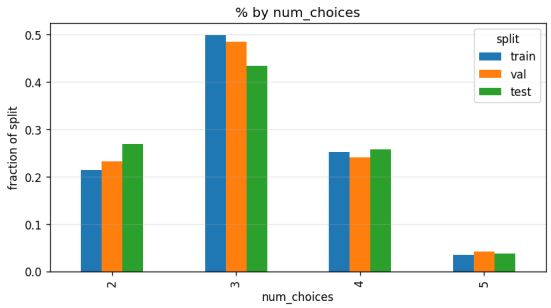

Figure 1: Distribution of `num_choices` across train, validation, and test splits. The test set contains noticeably more two-choice questions (+5.6 percentage points) and fewer three-choice questions (−6.5 points) than the training set. Subject and grade distributions (not shown) are nearly identical across all three splits.

2. **Cross-version model selection.** We compare versions by their best validation accuracy when deciding which adapter to submit to the test leaderboard.

3. **Error analysis substrate.** Because validation labels are available, we use validation predictions to compute per-slice accuracy (by `num_choices`, `subject`, gold answer index), confusion matrices, and qualitative error inspection. These inform the design of subsequent runs.

A consequence of this protocol is that one of our experimental conditions (Section 7) trains on TRAIN ∪ VAL rather than TRAIN. We report this clearly and note that for that condition the periodic "validation accuracy" becomes a memorization signal rather than a generalization estimate; we rely on test-leaderboard scores for that condition's evaluation.

3.4 Distribution Drift Between Splits

While the splits are well-matched on `subject` (within 1.5 percentage points across all three) and `grade` (within 1.6 points on every grade level), they differ measurably on `num_choices` (Figure 1). The test set has 27.0% two-choice questions, compared to 23.3% in val and 21.4% in train. Conversely, the test set has 43.5% three-choice questions, compared to 48.5% in val and 49.9% in train. Because two-choice questions admit higher accuracy than three- or four-choice questions for any model that is at least informative, this drift contributes a structural component to the validation-to-test accuracy gap that we observe consistently across our runs.

3.5 Slice Composition and the Five-Choice Specialty

The five-choice slice deserves separate discussion because it is uniquely concentrated. Of the 110 five-choice training examples, 109 are biology questions in the "Genes to traits" category, and 99 of those are explicitly Punnett-square problems requiring the model to compute offspring probabilities or ratios. The five-choice validation slice (44 rows) is 98% Punnett-square problems; the test slice (38 rows) is 92%. This concentration means that performance on the five-choice slice is effectively a measure of one specific multi-step reasoning capability: visual interpretation of a Punnett-square diagram, application of Mendelian genetics, and string matching of computed ratios against shuffled answer choices.

The three-choice slice (the largest, comprising roughly half of all examples) is also templated, though more diversely. The most common skills include comparing magnitudes of magnetic forces (251 train rows), classifying organisms by scientific name (213), comparing solution concentrations (192), and analyzing particle motion (168). These templates produce heavily-repeated choice strings: the literal answer text "`the magnitude of the magnetic force is the same in both pairs`" appears as a candidate answer in 251 distinct training questions; "`solution a`" and "`solution b`" each appear 192 times. This repetition has implications for choice-order augmentation, which we discuss in Section 7.

3.6 Data Integrity

We verified the following properties before training. All counts are zero unless otherwise specified.

Missing values. No row in any split has a missing value in `id`, `image_path`, `question`, `choices`, or `num_choices`. The training and validation sets additionally have no missing `answer` values. The optional `hint`, `lecture`, and `solution` fields are present for the majority of rows but are not required.

Schema consistency. For every row, `num_choices` matches the length of the parsed `choices` list, and the `answer` index is within $[0, \texttt{num\_choices})$.

Image leakage. No image filename appears in more than one split. The intersections TRAIN∩VAL,

TRAIN∩TEST, and VAL∩TEST are all empty. There are no within-split duplicate image filenames either.

Truncation risk. We use a maximum text-token budget of 1,280 throughout (Section 4). We measured the composed prompt length (lecture + hint + question + choices + solution where applicable) per row and found that only 2 of 3,109 training rows exceed 5,000 characters (approximately 1,250 tokens), with 0 such rows in val and 0 in test. Truncation under our token budget is therefore effectively absent.

Category coverage. The training set spans 55 distinct `category` values. The validation set contains 7 examples (0.7%) and the test set contains 5 examples (0.5%) drawn from categories that do not appear in train. We do not attempt special handling for these examples.

Answer-index frequency. The `answer` index is highly imbalanced: index 0 accounts for 36% of training labels and index 1 for 33%, while index 4 accounts for only 16 examples (0.5%) in train and 7 examples (0.7%) in val. This imbalance is consistent across splits and we do not perform inverse-frequency reweighting.

4 Method

We adapt SmolVLM-500M-Instruct (Marafioti et al., 2025), an instruction-tuned compact vision-language model, to the multiple-choice task using Low-Rank Adaptation (LoRA; Hu et al., 2021). This section describes the base model, the LoRA configuration used in our default training recipe, prompt construction, the training objective, and the inference procedure. The Ablations section (Section 7) varies one or two factors at a time relative to the default described here.

4.1 Base Model

SmolVLM-500M-Instruct combines a SigLIP vision tower (Zhai et al., 2023), a learned modality-projection layer, and a small autoregressive language-model decoder. We load the official Hugging Face checkpoint `HuggingFaceTB/SmolVLM-500M-Instruct` unmodified, including its tokenizer, image processor, and chat template. The vision tower and base language-model weights remain frozen throughout training; only the LoRA adapters are updated.

We configure the image processor with `longest_edge` $= 384$ pixels, the larger setting that fits within our T4 memory budget at the chosen batch and sequence-length settings. One ablation (Section 7) raises this to 512 to test the resolution-versus-batch-size tradeoff.

4.2 LoRA Adaptation

The default LoRA configuration injects trainable low-rank matrices of rank $r = 24$ with scaling factor $\alpha = 48$ (yielding effective scale $\alpha/r = 2$) into the four attention-projection modules of every transformer block in the language-model decoder: the query, key, value, and output projections (`q_proj`, `k_proj`, `v_proj`, `o_proj`). This produces $4{,}915{,}200$ trainable parameters, comfortably within the 5 M cap. We use LoRA dropout of 0.05 in the default recipe. Two versions (v2, v5) used 0.10, but in both cases the dropout change is confounded with other modifications; we do not attempt a clean dropout ablation. We do not apply LoRA to the vision tower, the modality projection, or the language-model embedding/head.

The Ablations section varies (i) the rank/alpha pair (rank 16, $\alpha = 32$ as the smaller alternative, yielding $3{,}276{,}800$ trainable parameters); and (ii) the set of target modules (MLP-only; mixed query/value plus MLP).

4.3 Prompt Construction

Each example is rendered into a single user-turn message containing the image, the supporting context, the question, and the candidate choices, then wrapped in the model's native chat template. We refer to this rendering as the **v5_baseline** prompt format. The text portion follows the schema (`<. . . >` fields are omitted if the source row's corresponding column is empty, and choice lines beyond `C` appear only for $k > 3$ examples) :

```
Lecture: <lecture_text>
Context: <hint_text>
Question: <question_text>
Choices:
A. <choice_0>
B. <choice_1>
C. <choice_2>
[D. <choice_3>]
[E. <choice_4>]
[Reasoning hint: <solution_text>]
Respond with a single letter from
[A, B, C, . . . ].
Answer:
```

The full text is concatenated with the image-token placeholder and passed through the chat-template formatter, producing a token sequence of the form `<image>\n<text>\n<assistant_header>`. The

assistant turn is empty: at training time the answer letter is appended as a single token; at inference time the model produces logits at the position immediately following the assistant header.

Chain-of-thought as input. For each training example whose source row contains a non-empty `solution` field, we include that solution as additional context with probability `cot_prob` $= 0.5$, inserted between the choices block and the response instruction as "Reasoning hint: <solution_text>". This setting is active during training only; the test split does not contain solution text (Section 3), so the model never sees it at inference. We refer to this as *CoT-as-input* to distinguish it from *generative CoT* training, in which the model is supervised to produce the reasoning trace itself evaluated as an ablation (Section 7).

The Ablations section additionally varies the prompt format itself: a more compact question-first variant and a raw-image variant that bypasses the chat template, both used to test sensitivity of the trained model to surface form.

4.4 Training Objective

We treat answer-letter prediction as single-token next-token classification. For each training example, the prompt is rendered as above and the gold answer letter (one of `A`, `B`, `C`, `D`, `E`, mapped from the integer answer index) is appended as a single token. Cross-entropy loss is computed at the position whose target is the gold letter; loss at all other positions is masked out.

In a causal-LM setup this corresponds to predicting the letter token from the assistant-header position. Getting this index correct in practice is non-trivial because the chat template emits multiple tokens after the user turn before the model's first generation slot, and an off-by-one here trains the model on an unrelated target. We verified the alignment by inspecting the loss-mask positions against the tokenized prompt for several examples before launching training, and we report on the consequences of getting this wrong in Section 7.

The Ablations section additionally evaluates two alternative objectives: (i) *answer-first generative CoT*, in which the model is supervised to emit the answer letter followed by a reasoning trace, with cross-entropy applied to the answer and reasoning tokens; and (ii) *reasoning-first generative CoT*, in which the model emits the reasoning first and then the answer letter.

4.5 Inference

At inference time we render the prompt identically to training (omitting the `Reasoning:` block, since `solution` is unavailable on the test split), perform a single forward pass with `fp16` autocast, and read the logits at the last attended position. We restrict attention to the five letter-token IDs $L = \{\texttt{A}, \texttt{B}, \texttt{C}, \texttt{D}, \texttt{E}\}$ by setting all other logits to $-\infty$, then additionally mask out positions $i \geq k$ for an example with k choices, so that a 3-choice question can only resolve to A, B, or C. The predicted answer index is the argmax over the remaining unmasked letter logits. The procedure is greedy, deterministic, and does not generate any tokens beyond the single classification step.

The Ablations section evaluates two inference-pipeline interventions on top of this base procedure: (i) substituting an exponential moving average (EMA) of the LoRA weights for the live weights at evaluation time; and (ii) test-time augmentation (TTA) by averaging logits across multiple choice-order permutations of the same example.

5 Experimental Setup

5.1 Compute and Libraries

All training and inference were performed on a single NVIDIA Tesla T4 (16 GB) provided by Google Colab Free, using PyTorch with `fp16` autocast for both forward and backward passes; optimizer state and gradient-accumulation buffers are kept in fp32 by a `torch.amp.GradScaler`. We used `transformers 4.49.0` and `peft 0.13.2`. Per-run wall-clock training time was typically between 22 minutes and 1.8 hours, with one substantial outlier: v12 (reasoning-first generative CoT) consumed 30.4 hours, an order of magnitude more than other runs, due to a fundamental difference in the validation protocol it required (Section 9). Peak GPU memory during training ranged from 3.66 to 6.66 GB across runs.

5.2 Optimization

We optimize with AdamW (PyTorch implementation) on the LoRA-adapter parameters only, with `weight_decay` $= 0$. The learning-rate schedule is cosine decay with linear warmup over the first 3% of total optimizer steps (`warmup_ratio` $= 0.03$), implemented via `get_cosine_schedule_with_warmup` from `transformers`. The base learning rate is 1×10^{-4}

in our default recipe; one ablation (Section 7) lowers it to 5×10^{-5} paired with an extra training epoch. Gradient norms are clipped at 1.0 after the scaler's `unscale_` step.

The micro-batch size is 2 examples per step with gradient accumulation over 8 micro-batches, yielding an effective optimization batch of 16 examples per gradient step. Tokenized inputs are right-padded and truncated to a maximum of 1,280 text tokens (excluding the image-token placeholder); as established in Section 3.6, only $2/3{,}109$ training rows exceed this budget, so truncation is effectively absent.

5.3 Training Loop

We train for between 2 and 5 epochs depending on version. Within each epoch the data loader uses a fresh PyTorch generator seeded as `SEED + epoch`, with shuffling enabled and `drop_last=True`, which guarantees that any reported run can be reproduced step-for-step from the seed. We evaluate on the validation set every 50 optimizer steps and save the LoRA adapter whose validation accuracy is the running maximum (best-checkpoint selection). At each evaluation step we additionally append a record to a per-run `metrics.json` log containing the step number, smoothed loss, current learning rate, gradient norm, and validation accuracy.

5.4 Evaluation Protocol

The evaluation metric is single-choice accuracy: the predicted answer index (argmax over the masked letter logits described in Section 4.5) is compared to the gold answer index, with each example contributing equally to the mean. Validation accuracy is computed over all 1,048 validation examples. Test accuracy is computed by submitting predictions for all 1,008 test examples to the public-leaderboard scorer; we do not have access to the private leaderboard or to its scoring schedule, so all test numbers reported in this paper are public-leaderboard accuracies. Where we report "best validation accuracy" for a run, we mean the maximum across all 50-step evaluation points for that run.

5.5 Version Tracking and Reproducibility

Each training configuration is identified by a version tag (`v1`–`v13`) corresponding to a self-contained training notebook. Each version writes its outputs to a separate directory containing the LoRA adapter, the optimizer, scheduler, and gradient-scaler states, the random-number-generator state (Python, NumPy, and CUDA), and a `metrics.json` record. The default random seed is 42; one ablation (v7) repeats v4's configuration with seed 1337 to provide a single-seed sensitivity estimate.

Before any training run, we verified that no image filename appears in more than one split, that every row has the required fields (`id`, `image_path`, `question`, `choices`, `num_choices`), and that `num_choices` matches the length of the parsed `choices` list (Section 3.6). The combination of fixed seed, per-step logged metrics, and version-tagged outputs allows any reported run to be reconstructed by re-executing the matching notebook on equivalent hardware.

6 Results

We report public-leaderboard test accuracy for all thirteen training runs, plus two inference-only ablations (v9-ablation, v10-ablation) on saved adapters and two logit-averaged ensembles. Best validation accuracy and zero-shot validation accuracy under each run's prompt format are reproduced from Section 5 for context.

6.1 Headline Result

Our strongest single-adapter submission, denoted v9-ablation, achieves **0.831** test accuracy. The underlying adapter is the v9 LoRA adapter, trained for 5 epochs on TRAIN $\cup$ VAL with rank-24 attention-projection LoRA; the suffix "ablation" indicates that this submission used the deliberately minimal inference recipe of Section 4.5: live LoRA weights, no exponential moving averaging, no test-time augmentation.

For comparison, the same adapter under the original v9 inference recipe that additionally applied an exponential moving average (EMA) over the LoRA weights and test-time augmentation (TTA) by choice-order permutation scored only 0.805 on the public test leaderboard. The 2.6-percentage-point gap between these two submissions, both derived from a single trained adapter, is the largest inference-side effect in our study and is analyzed in Section 7.

6.2 Per-Version Summary

Table 1 reports validation and test accuracy for every submitted version. All thirteen LoRA-fine-tuned runs achieve validation accuracy between

0.717 and 0.818 and test accuracy between 0.757 and 0.831, well above both the all-zero-prediction floor of 0.360 and the un-adapted zero-shot floor of 0.508 for the v5_baseline prompt format.

Figure 2 shows zero-shot vs. best-LoRA validation accuracy per version side-by-side; Figure 3 shows the validation-accuracy trajectory during training for every run on a shared step axis. The visual story is dominated by two clusters: the v1–v10, v13 cluster, which tops out between 0.75 and 0.82 on validation, and the v11–v12 generative-CoT cluster, which is uniformly 5–6 points lower. v9 sits at the top of the trained-adapter cluster but, as discussed below, this is partly a within-distribution artifact.

6.3 The Validation–Test Gap

A consistent feature of our results is that test accuracy exceeds best validation accuracy by roughly 3–4 percentage points across all TRAIN-only runs. For v4, the gap is $0.785 \rightarrow 0.829$ ($+0.044$); for v3, $0.773 \rightarrow 0.811$ ($+0.038$); for v8, $0.782 \rightarrow 0.823$ ($+0.041$); for v13, $0.792 \rightarrow 0.823$ ($+0.031$). The direction is the same on every TRAIN-only run.

This gap has a structural explanation grounded in Section 3.4: the test split has a higher fraction of two-choice questions (27.0%) than validation (23.3%) and a lower fraction of three-choice questions (43.5% vs. 48.5%). Since two-choice items admit higher accuracy than three-choice items for any informative model, this distributional shift contributes a few percentage points of the observed gap independent of cross-split generalization quality. We do not attempt a quantitative decomposition because doing so would require a per-slice accuracy estimate on the test set, which we cannot compute without test labels. Instead, we interpret the gap as primarily distributional rather than evidence of a deeper test-set effect.

For v9, the gap reverses sign: best val 0.818 exceeds test 0.805 (or, with the cleaner inference recipe, the test goes back up to 0.831). This is consistent with v9's val accuracy being partly memorization, since VAL $\subseteq$ training data for that run.

7 Ablations

We organize our ablations around the variable hooks established in Section 4, with one cluster per hook. Table 2 summarizes all pairwise test-accuracy deltas; the prose below explains each cluster and identifies what we believe is the most informative finding in each.

7.1 Capacity and Training Duration

Increasing the rank from 16 to 24 while scaling α proportionally from 32 to 48 to preserve the effective scale $\alpha/r = 2$ produces a $+0.008$ test improvement (v1 $\rightarrow$ v3). This comparison also includes the introduction of CoT-as-input, which we treat as a confound since v1 was the only run without CoT-as-input. Extending training from 3 to 5 epochs (v3 $\rightarrow$ v4) provides an additional $+0.018$ gain. Together, these two changes contribute $+0.026$ test points, matching the full performance range observed across our other single-adapter variants. At this scale, increasing rank and training longer are the primary factors that raise the trained-adapter ceiling, whereas most other interventions either regressed performance or produced negligible changes.

7.2 LoRA Target Topology

We compared three target-module configurations: attention projections (q/k/v/o, the default), MLP layers only (v8, at rank 16), and a mixed configuration with query/value attention plus MLP layers (v13, at rank 16, additionally combined with self-generated image-caption augmentation). Both alternatives regress by -0.006 test points relative to v4. The MLP-only configuration (v8) is the more striking of the two: it achieves comparable accuracy (0.823 vs. 0.829) with 33% fewer trainable parameters (3.28 M vs. 4.92 M), suggesting that this task does not specifically require attention-projection adaptation. We do not have a clean ablation isolating image captions from the LoRA-topology change in v13, so we cannot attribute v13's regression to either factor alone.

7.3 Prompt Format

We tested two alternative prompt renderings against the chat-templated v5_baseline. The compact question-first variant (v6) places the question and choices before the lecture and hint, drops the chat template's role markers, and uses a shorter response instruction. It regresses by -0.036 test points. The raw-image format (v10), which omits the chat template entirely and feeds the image and a plain-text instruction directly, regresses by only -0.004 test points (using v10-ablation to control for inference-pipeline differences).

A more interesting pattern emerges when comparing prompt-format effects before adaptation

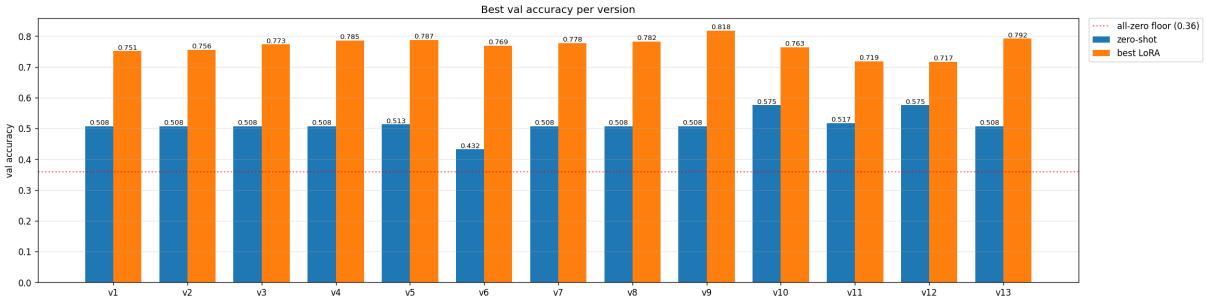

Figure 2: Zero-shot and best validation accuracy for each training run. Dotted line marks the all-zero-prediction floor (0.36).

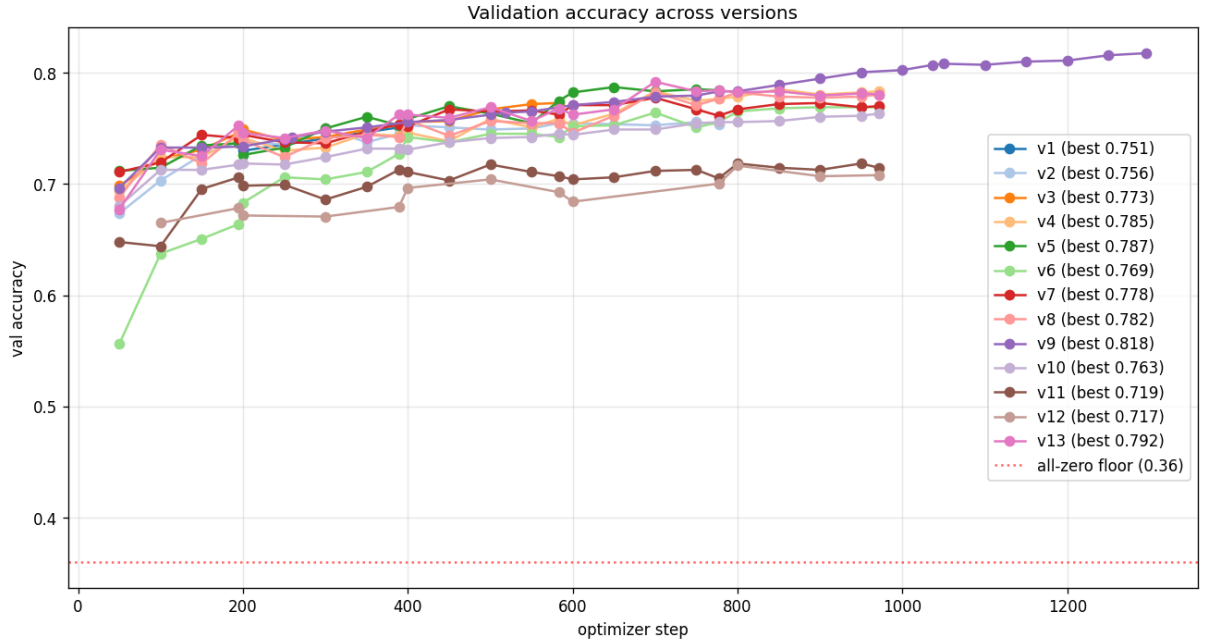

Figure 3: Validation accuracy across all thirteen training runs, plotted against optimizer step. v9 (trained on TRAIN ∪ VAL) reaches the highest curve but is not directly comparable to the others; v11 and v12 (generative chain-of-thought) form the lowest cluster.

with those observed after adaptation. The compact format used in v6 achieves a zero-shot validation accuracy of only 0.432, substantially below the 0.508 obtained by v5_baseline, whereas the raw-image format in v10 reaches 0.575. However, after 5 epochs of LoRA fine-tuning, the resulting test accuracies converge to 0.793, 0.829, and 0.825 for v6, v4, and v10-ablation respectively, corresponding to a spread of less than 4 points despite an initial zero-shot gap of roughly 14 points. Prompt-format differences therefore appear to diminish substantially after LoRA fine-tuning at this scale, suggesting that the adapter learns to interpret whichever surface representation it is trained on.

7.4 Training Data Composition

Training on TRAIN ∪ VAL (4,157 examples) versus TRAIN alone (3,109 examples) gains only +0.002 test points (v4 → v9-ablation). The marginal value of an additional 1,048 examples drawn from the same distribution is essentially nil at five epochs of training. We interpret this as evidence that the LoRA adapter has saturated on the kind of supervision the dataset provides: additional examples do not unlock new patterns, only repeat existing ones. This finding shapes our recommendation in Section 9: at this scale and dataset size, VAL is more valuable kept aside as a model-selection signal than folded into training.

ver	key change	zero-shot val	best val	test
v1	rank 16, no CoT-as-input, 2 epochs	0.508	0.751	0.803
v2	+ CoT-as-input, 4 epochs, lr 5×10^{-5}	0.508	0.756	0.795
v3	rank 24, $\alpha = 48$, 3 epochs	0.508	0.773	0.811
v4	5 epochs (default recipe)	0.508	0.785	**0.829**
v5	image edge 512, choice augmentation, dropout 0.10	0.513	0.787	0.813
v6	compact question-first prompt	0.432	0.769	0.793
v7	seed 1337	0.508	0.778	0.803
v8	MLP-only LoRA at rank 16	0.508	0.782	0.823
v9	v4 + TRAIN ∪ VAL, EMA, TTA	0.508	0.818	0.805
v9-ablation	v9 adapter, no EMA, no TTA	—	—	**0.831**
v10	raw-image format (no chat template)	0.575	0.763	0.805
v10-ablation	v10 adapter, no EMA, no TTA	—	—	0.825
v11	answer-first generative CoT	0.517	0.719	0.767
v12	reasoning-first generative CoT	0.575	0.717	0.757
v13	captions + mixed q/v + MLP LoRA	0.508	0.792	0.823
v4 + v5 ensemble (logit average)	—	—	0.807	
v4 + v9 ensemble (logit average)	—	—	0.827	

Table 1: Per-version validation and test accuracy. Test accuracy is the public-leaderboard score. "Zero-shot val" is the un-adapted SmolVLM-500M-Instruct's validation accuracy under that version's prompt format. "Best val" is the maximum across the periodic 50-step evaluations during training. v9 was trained on TRAIN ∪ VAL, so its best-val column includes in-training data and is not directly comparable to the others; we report it for completeness. Bold marks the strongest TRAIN-only single-adapter submission (v4) and the headline submission (v9-ablation).

7.5 Training Objective

A development-time finding on letter-CE supervision. Before discussing alternative objectives, we note one finding from the canonical letter-CE objective itself. Causal language modeling places the token-prediction loss at position i to predict the token at position $i{+}1$. For our chat-templated prompt ending in "Answer:", the assistant header emits multiple tokens before the model's first generation slot, so the position whose logits should yield the gold answer letter is several tokens before the letter itself rather than the obvious "last token" position. An off-by-one here trains the model on the wrong target, and the resulting failure is silent: training loss decreases, gradients look healthy, but validation accuracy never rises above the all-zero floor. We caught this during initial training-run validation by inspecting the loss-mask positions against the tokenized prompt; all results in this paper use the corrected position. We highlight the issue here because the failure mode is easy to miss without explicit verification of the supervision alignment.

Generative chain-of-thought training. Two ablations replace the single-token letter-CE objective with a generative objective: v11 supervises the model to emit the answer letter followed by a reasoning trace (*answer-first*), and v12 supervises the reverse ordering (*reasoning-first*). Both regress substantially: -0.062 and -0.072 test points, respectively. These are the largest negative deltas in our study.

The headline interpretation, supported by per-example confusion analysis in Section 8, is that the failure is diffuse degradation rather than structural collapse. v11 and v12 retain v4's pattern of off-diagonal confusion, particularly the dominant $0\leftrightarrow1$ and $1\leftrightarrow2$, complete abandonment of index 4, but they lose 15–30 correct predictions in every gold class. The model does not develop a new failure mode under generative supervision; it loses sharpness on the failure modes it already had. We read this as consistent with prior reports that chain-of-thought benefits emerge at much larger model scale (Wei et al., 2023, 2022): at 500 M parameters, supervising the model on its own reasoning does not appear to improve the underlying reasoning, while it does cost some discriminative precision on the letter-prediction task.

7.6 Inference Pipeline

The largest inference-side effect in our study comes from running the same trained adapter under two different inference recipes. Our original v9 inference recipe applied (i) an exponential moving average of the LoRA weights with decay 0.999, and (ii) test-time augmentation by averaging logits across four choice-order permutations of each example. Removing both interventions (v9 $\rightarrow$ v9-ablation)

Ablation pair	Δ test
Capacity and training duration	
v1 → v3 (rank 16 → 24, α 32 → 48)	+0.008
v3 → v4 (3 → 5 epochs)	+0.018
v1 → v4 (combined)	+0.026
LoRA target topology	
v4 → v8 (q/k/v/o → MLP-only at rank 16)	−0.006
v4 → v13 (q/k/v/o → q/v + MLP, + captions)	−0.006
Prompt format	
v4 → v6 (chat-templated → compact q-first)	−0.036
v4 → v10-abl (chat-templated → raw image)	−0.004
Training data	
v4 → v9-abl (TRAIN → TRAIN ∪ VAL)	+0.002
Training objective	
v4 → v11 (letter CE → answer-first CoT)	−0.062
v4 → v12 (letter CE → reasoning-first CoT)	−0.072
Inference pipeline	
v9 → v9-abl (drop EMA & TTA)	+0.026
v10 → v10-abl (drop EMA & TTA)	+0.020
v10 (live + TTA) → v10-abl (live)	±0
Seed	
v4 (seed 42) → v7 (seed 1337)	−0.026

Table 2: Ablation test-accuracy deltas. Positive Δ means the rightward configuration scores higher on the public test leaderboard.

gains +0.026 test points; the same change on the v10 adapter gains +0.020.

To attribute the gain, we ran a third inference pass on the v10 adapter with TTA enabled but EMA disabled. The result was identical to v10-ablation to the displayed precision. **TTA contributes nothing on this task.** Choice-order permutation averaging only helps when the model's confidence depends on the order in which choices are presented. We found our model to be sufficiently confident on most test rows that the four permuted forward passes produce nearly identical logits and average to the same prediction.

The full 2.6-point cleanup gain therefore comes from removing EMA. The mechanism is straightforward: EMA shadow weights are computed as $w_{shadow} \leftarrow 0.999 \cdot w_{shadow} + 0.001 \cdot w_{live}$, giving an effective averaging window of roughly 1,000 optimizer steps. Our v9 run lasted only 1,300 steps total. The shadow weights at end of training therefore retained substantial mass on the still-improving early-training weights rather than reflecting the converged adapter. Figure 4 shows that v9's live LoRA weights gain roughly 0.12 validation-accuracy points between step 50 and step 1,300; an EMA window of comparable length to the run necessarily averages over much of that improvement. EMA can be a useful regularizer when the run is long compared to its averaging window; ours was not.

The practical lesson is that "standard" inference-time interventions can silently regress at the run lengths typical of small-scale parameter-efficient fine-tuning. We caught this only because we kept saved adapters and re-ran inference; an experimental loop that always submits the EMA-augmented prediction would never have observed the live-weight improvement.

7.7 Seed Sensitivity

We ran one cross-seed comparison: v7 reproduces v4's configuration with seed 1337 instead of 42, yielding a test-score change of −0.026 points (v4 0.829 vs. v7 0.803). With a single seed pair we cannot distinguish run-to-run variance from a small genuine effect of seed choice on this dataset, and reporting a confidence interval would require many more seeds than our compute budget allowed. We highlight this gap as a lower bound on the noise floor of any single-seed result in this study, which serves as a caveat for the smaller deltas in Table 2 (notably the ±0.006 topology results and the +0.002 training-data result, both of which are within the seed gap).

8 Error Analysis

We analyze where the trained model fails through two views: the per-slice validation breakdown of our default adapter (v4), and confusion-matrix comparisons between the letter-CE objective (v4) and the two generative-CoT objectives (v11, v12). All analyses use validation-set predictions, since test labels are not available.

8.1 Confusion Structure of the Default Adapter

Figure 5 shows the validation confusion matrix for the v4 final-epoch adapter (overall accuracy 0.783). The confusion structure is striking for what it is and what it is not. The dominant off-diagonal entries are near-neighbor confusions: 44 instances where gold 1 was predicted as 0, 40 of gold 0 predicted as 1, 32 of gold 1 predicted as 2, 31 of gold 0 predicted as 2, and 29 of gold 2 predicted as 0. Roughly symmetric off-diagonals on the upper-left 3×3 block account for over 70% of total errors.

The matrix is also striking for two near-zero rows. Of 7 validation examples whose gold answer

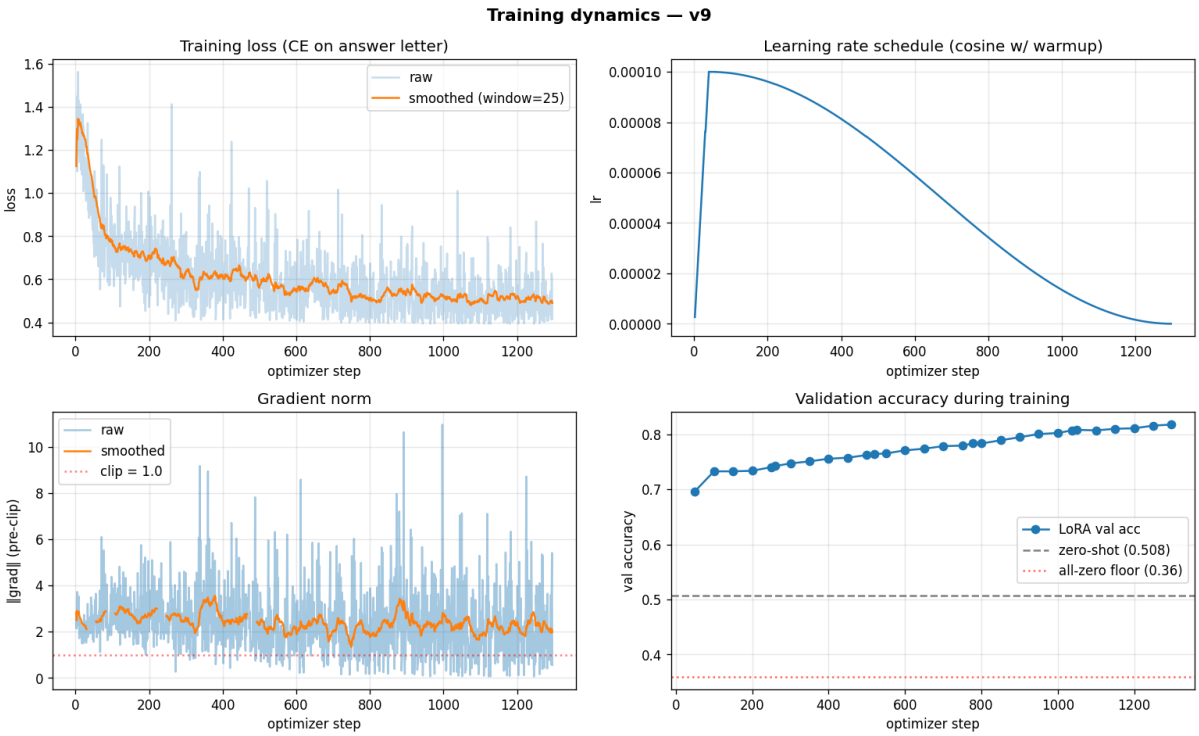

Figure 4: Training dynamics for v9 (trained on TRAIN ∪ VAL, 5 epochs). *Top-left*: training cross-entropy loss with 25-step smoothing. *Top-right*: cosine learning-rate schedule with linear warmup. *Bottom-left*: pre-clip gradient norm (clip threshold 1.0). *Bottom-right*: validation accuracy at the periodic 50-step evaluation points. The validation curve is partly an in-training measurement; see Section 6. The $\sim$0.12-point climb between step 50 and step 1,300 is the source of EMA's underperformance: a decay of 0.999 averages over a window of comparable length to the run.

is index 4, the model correctly predicts none, and produces an index-4 prediction only once across the entire validation set. Of 75 examples whose gold answer is index 3, the model recovers 57 (accuracy 0.76), with relatively small off-diagonal mass. The matrix suggests that the model has learned the first three answer slots reliably, the fourth slot only moderately well, and the fifth slot almost not at all, a pattern that appears to arise from a structural issue discussed next.

8.2 Slice-Level Disparities

Figure 6 decomposes v4's validation accuracy along four axes: `num_choices`, gold answer index, `subject`, and the predicted-versus-gold answer-index distribution. Three disparities deserve discussion, plus one calibration observation.

Five-choice questions collapse to chance. The model achieves 0.82, 0.78, and 0.85 accuracy on two-, three-, and four-choice questions respectively, but only **0.25** accuracy on five-choice questions ($n = 44$). As established in Section 3.5, the five-choice slice is overwhelmingly templated: 98% of validation five-choice rows are Punnett-square problems requiring the model to (i) interpret a square diagram showing parental allele combinations, (ii) apply Mendelian inheritance rules to compute offspring genotype or phenotype proportions, and (iii) string-match the computed ratios against a list of shuffled answer choices that often share textual content (e.g., "`1/4`", "`2/4`", "`3/4`"). This is effectively a single-capability slice, and the model has not acquired that capability under our training recipe. The slice is small enough ($\sim$ 4% of every split) that it does not dominate aggregate accuracy, but it is large enough that closing this gap would meaningfully improve test scores.

Index-4 gold answers are never correctly predicted. The five-choice collapse explains the index-4 result mechanically: index 4 exists only in five-choice questions. Of v4's 7 index-4 validation rows, the model recovers none, and it produces an index-4 prediction only once across all 1,048 validation examples. We interpret this as evidence that the letter-prediction logit corresponding to index 4 never becomes large enough to be selected, indicating a calibration failure on a data slice that is almost absent from the training distribution (16

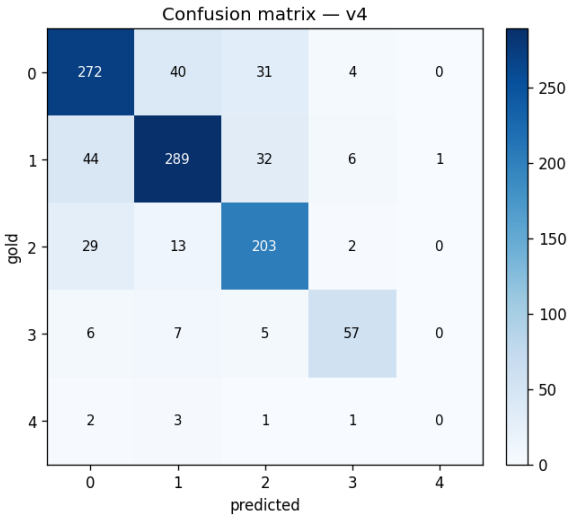

Figure 5: Validation confusion matrix for the v4 final-epoch adapter. Rows are gold answer indices (0–4); columns are predicted indices. Diagonal sum is $821/1{,}048 = 0.783$.

index-4 examples out of 3,109 training samples, or approximately 0.5%).

Language-science questions underperform. By subject, v4 reaches 0.89 on social-science questions ($n = 243$), 0.76 on natural-science questions ($n = 777$), and 0.61 on language-science questions ($n = 28$). The language-science slice is small enough that we cannot draw strong conclusions, but the gap is large enough to flag. With 28 examples and high-choice questions, 0.61 accuracy sits within range of chance for some sub-slices. Future work might consider mild upsampling of the smallest `subject` slice during training.

Marginal calibration is otherwise close to gold. Comparing predicted and gold answer-index distributions in the bottom-right panel of Figure 6, predicted counts at indices 0 through 4 are 353, 352, 272, 70, 1 versus gold 347, 372, 247, 75, 7. The model over-predicts index 2 by 25 examples and under-predicts index 1 by 20. The most extreme miscalibration is the index-4 collapse described above. The overall pattern is a model whose predictive marginals are close to the input distribution, with one severe slice failure showing up as an isolated calibration gap rather than a global bias.

8.3 Generative-CoT Failure Mechanism

Section 7.5 reported that the two generative-CoT variants (v11 answer-first, v12 reasoning-first) regress by 0.062 and 0.072 test points relative to the letter-CE baseline, and asserted without proof that the failure is diffuse degradation rather than a structural collapse. Figures 7 and 8 provide the evidence for that claim.

The off-diagonal pattern is preserved. Both v11 and v12 retain v4's dominant pattern of $0 \leftrightarrow 1$ and $1 \leftrightarrow 2$ near-neighbor confusion, with the same upper-left 3×3 block carrying most of the error mass. Both retain v4's complete failure on index 4: zero correct predictions out of seven gold rows for both variants. v11 retains v4's count of 57 correct gold-3 predictions; v12 drops only to 53. There is no qualitative change in the type of error the model makes.

What changes is per-class diagonal mass. Each gold-class diagonal in v11 and v12 sits below the corresponding v4 diagonal: index 0 loses 29 correct predictions in v11 (v4 272 $\rightarrow$ v11 243) and 15 in v12 (v4 272 $\rightarrow$ v12 257); index 1 loses 27 and 29; index 2 loses 16 and 31; index 3 loses 0 and 4. The lost predictions are spread across the same off-diagonal cells that v4 was already losing, with each cell gaining a small additional count. v12 additionally shifts predictive mass toward index 0: it predicts 0 for 373 examples versus v4's 353 and v11's 359. We interpret this as reasoning-first supervision drawing the model toward the first letter token in its own generated trace, but the effect is small relative to the overall regression.

Interpretation. Generative chain-of-thought supervision at 500 M parameters did not produce a qualitatively new failure mode: the model did not collapse toward a single index, generate malformed outputs, or lose its existing near-neighbor confusion structure. It reduced discriminative sharpness on the failure modes the model already had. We read this as the predicted outcome of training a small model to imitate reasoning traces it cannot internally simulate: the supervision adds noise to the answer-prediction signal without supplying additional inductive bias for the underlying task.

9 Limitations

Model scale. All findings are reported for the SmolVLM-500M-Instruct base model under a 5 M trainable-parameter LoRA cap. Prior work suggests that several of our negative findings, particularly the failure of generative chain-of-thought

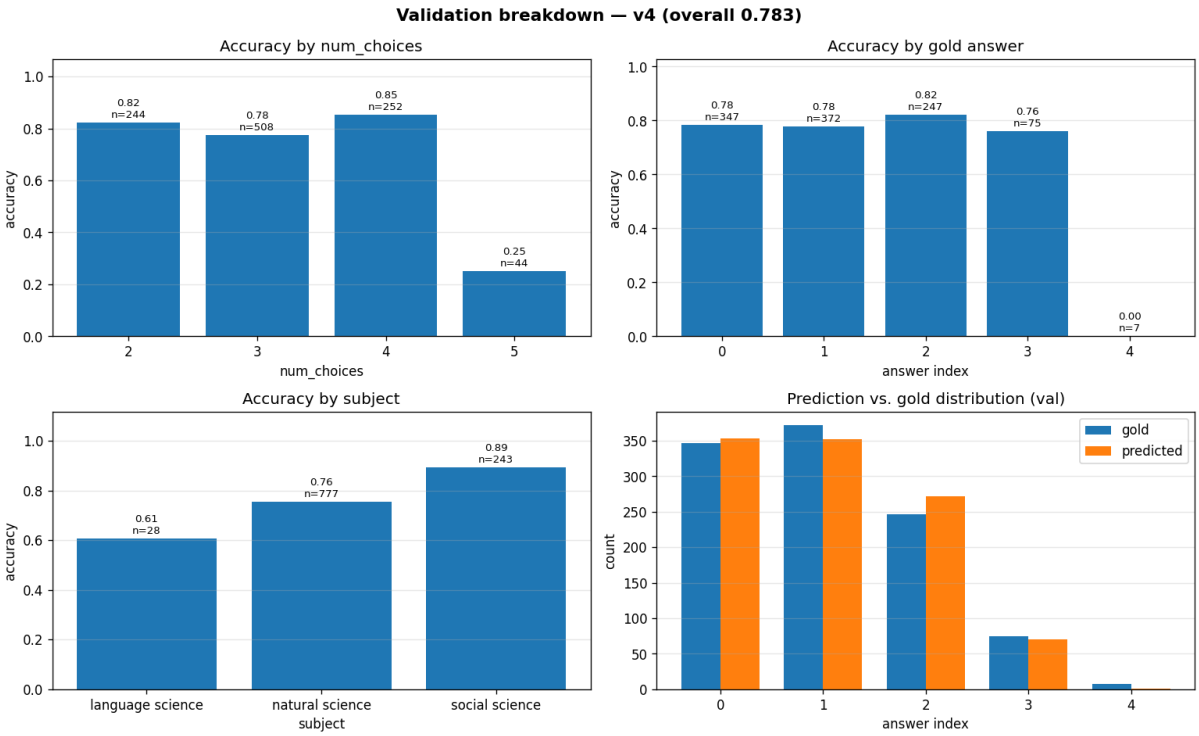

Figure 6: Validation-set breakdown for the v4 final-epoch adapter (overall 0.783). *Top-left*: accuracy by `num_choices`; the five-choice collapse is the most visible feature. *Top-right*: accuracy by gold answer index. *Bottom-left*: accuracy by `subject`. *Bottom-right*: predicted vs. gold answer-index distribution across the full validation set.

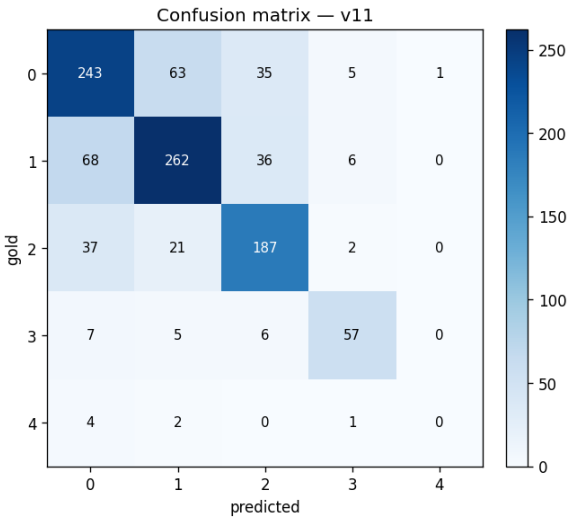

Figure 7: Validation confusion matrix for v11 (answer-first generative CoT). Compare with Figure 5.

training and the limited difference between train-only and train+validation performance, may behave qualitatively differently at substantially larger model scales (**?**Wei et al., 2022). We do not assess whether the levers we identify (capacity, duration) and the warnings we issue (EMA window length, prompt-format fragility under fine-tuning) hold past a base-model size that fits on a single T4. The most useful followup is a replication at the 2–7 B parameter scale, which we did not have compute for.

Cost asymmetry between training objectives. Reasoning-first generative CoT (v12) required validation accuracy to be measured by full text generation rather than next-token letter logits, because the prompt no longer terminated at "`Answer:`". Each periodic validation pass involved $\sim$1,000 calls to `model.generate(max_new_tokens=250)`, taking on the order of 30 minutes per pass. Combined with longer test inference and the longest-running training session of any of our experiments, v12's wall-clock cost was roughly an order of magnitude higher than other versions. We note this as a practical limitation: reasoning-first CoT cannot be cheaply ablated against single-token baselines because the validation protocols themselves differ in cost. Our negative result on generative CoT is therefore based on a single run per ordering rather than a multi-seed estimate.

Single-seed noise floor. Outside the v4 vs. v7 pair, every ablation in our study is a single-seed

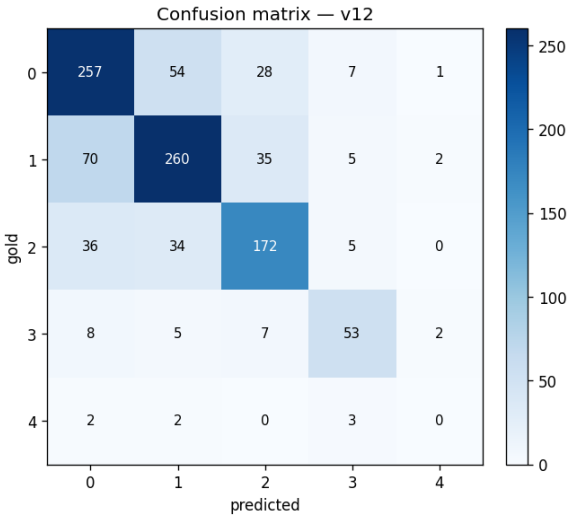

Figure 8: Validation confusion matrix for v12 (reasoning-first generative CoT). Compare with Figure 5.

comparison. The v4–v7 gap of -0.026 test points from a single seed change establishes a lower bound on run-to-run variance. Several of the smaller deltas in Table 2 fall within this noise floor: the topology results (± 0.006), the training-data result ($+0.002$), and the raw-image prompt-format result (-0.004). We are confident in the larger effects (capacity/duration $+0.026$, chat-templated vs. compact prompt format -0.036, training objective -0.062 to -0.072, inference cleanup $+0.020$ to $+0.026$), but caution against drawing strong conclusions from any individual delta smaller than ~ 0.03 test points.

Public-leaderboard scores only. All test numbers reported in this paper are public-leaderboard accuracies. The course-organized competition also maintained a private leaderboard, used for final grading, that we do not examine. Our model-selection decisions were therefore taken with respect to a strict subset of the eventual evaluation, and the public-leaderboard ranking may not faithfully reflect held-out generalization in the way a true held-out split would.

Dataset scope. Findings are restricted to the course-provided ScienceQA-derived benchmark: English-language, multiple-choice, image-grounded, with a majority of examples in natural science. Conclusions about LoRA capacity behavior, prompt-format fragility, and generative-CoT regression should not be assumed to transfer to open-ended generation, multilingual settings, or single-turn multimodal tasks without an answer-letter scaffolding.

10 Conclusion

We presented a parameter-efficient fine-tuning study of SmolVLM-500M-Instruct on a visual multiple-choice science QA task, conducted under a hard budget of 5 M trainable parameters and a single Tesla T4 GPU. Across thirteen training runs and fourteen pairwise ablations, the largest single positive contribution to test accuracy came from joint scaling of LoRA rank and training duration ($+0.026$), with a final single-adapter score of 0.831 on the public test leaderboard. Of equal practical importance is a negative finding: a deliberately "standard" inference recipe with EMA and TTA cost the same trained adapter 0.026 test points, an effect we trace to an EMA averaging window comparable in length to the entire training run. Generative chain-of-thought training reduced performance by 0.062 to 0.072 points and produced a confusion structure characterized by diffuse degradation of the letter-level cross-entropy baseline rather than collapse into a qualitatively new failure mode, consistent with prior findings that chain-of-thought benefits tend to emerge only at substantially larger model scales.

For practitioners working at this base-model size, our recommendations are to (i) prioritize rank and epoch budget over architectural creativity in the LoRA target set, (ii) retain saved adapters for direct re-evaluation under multiple inference recipes rather than committing to one polish layer at training time, and (iii) treat any single-seed test-score difference smaller than ~ 0.03 as within the noise floor of comparable-scale runs. We release configuration, training metrics, and per-version analysis to support replication.

References

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. Qlora: Efficient finetuning of quantized llms. *Preprint*, arXiv:2305.14314.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *Preprint*, arXiv:2106.09685.

Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. 2023. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. *Preprint*, arXiv:2301.12597.

Xiang Lisa Li and Percy Liang. 2021. Prefix-tuning: Optimizing continuous prompts for generation. *Preprint*, arXiv:2101.00190.

Haotian Liu, Chunyuan Li, Yuheng Li, and Yong Jae Lee. 2024a. Improved baselines with visual instruction tuning. *Preprint*, arXiv:2310.03744.

Shih-Yang Liu, Chien-Yi Wang, Hongxu Yin, Pavlo Molchanov, Yu-Chiang Frank Wang, Kwang-Ting Cheng, and Min-Hung Chen. 2024b. Dora: Weight-decomposed low-rank adaptation. *Preprint*, arXiv:2402.09353.

Pan Lu, Swaroop Mishra, Tony Xia, Liang Qiu, Kai-Wei Chang, Song-Chun Zhu, Oyvind Tafjord, Peter Clark, and Ashwin Kalyan. 2022. Learn to explain: Multimodal reasoning via thought chains for science question answering. *Preprint*, arXiv:2209.09513.

Andrés Marafioti, Orr Zohar, Miquel Farré, Merve Noyan, Elie Bakouch, Pedro Cuenca, Cyril Zakka, Loubna Ben Allal, Anton Lozhkov, Nouamane Tazi, Vaibhav Srivastav, Joshua Lochner, Hugo Larcher, Mathieu Morlon, Lewis Tunstall, Leandro von Werra, and Thomas Wolf. 2025. Smolvlm: Redefining small and efficient multimodal models. *Preprint*, arXiv:2504.05299.

Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. 2022. Emergent abilities of large language models. *Preprint*, arXiv:2206.07682.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2023. Chain-of-thought prompting elicits reasoning in large language models. *Preprint*, arXiv:2201.11903.

Xiang Yue, Yuansheng Ni, Kai Zhang, Tianyu Zheng, Ruoqi Liu, Ge Zhang, Samuel Stevens, Dongfu Jiang, Weiming Ren, Yuxuan Sun, Cong Wei, Botao Yu, Ruibin Yuan, Renliang Sun, Ming Yin, Boyuan Zheng, Zhenzhu Yang, Yibo Liu, Wenhao Huang, and 3 others. 2024. Mmmu: A massive multi-discipline multimodal understanding and reasoning benchmark for expert agi. *Preprint*, arXiv:2311.16502.

Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. 2023. Sigmoid loss for language image pre-training. *Preprint*, arXiv:2303.15343.

A Code Repository

All code is available at: https://github.com/Asai-Yume/ECE-GY-7123-Final

The repository contains the complete iterative development history, organized as follows:

- `notebooks/` — All notebooks used for training
- `scripts/` — Adapter key remapping utility
- `figures/` — Figures used in the report and data analysis
- `report/` — PDF of the final writeup

All versions from v1–v13 are included for completeness and to document the debugging process that led to the adapter key mismatch discovery. Each notebook is self-contained and includes setup instructions via Google Drive.